COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010/ Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	September 8, 2026	**DATE SUBMITTED:**	September 9, 2026
NAME:	Carl Andrei D. Torres	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 7.1 **Hands-On Activity 7.1 Sorting Algorithms**

1. PROCEDURE OUTPUT

ILO A

Code + Console Screenshot

```
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sortingAlgo.h"

const int max_size = 100;

void displayArray(const int arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}

int main() {
    int dataset[max_size];

    srand(time(0));

    for(int i = 0; i < max_size; i++) {
        dataset[i] = rand() % 100;
    }

    std::cout << "Original Array:\n";
    displayArray(dataset, max_size);
```

```
Original Array:
48 1 36 71 8 92 1 91 24 86 22 0 76 4 91 61 76 38 75 11 69 56 21 89 73 36 13 40 96 22 21 72 63 76 16 88 56 40 55 10 21 42 27 2
2 85 18 34 13 84 50 57 69 11 48 69 11 29 78 89 32 48 89 32 92 90 67 39 80 81 48 33 62 33 20 48 94 21 31 19 47 1 85 78 46 42 9
5 70 46 6 77 82 97 66 34 48 58 74 61 0 8
```

Observations

In this code and output, I implement the libraries for implementing the random variables in <cstdlib> and <time.h> for formatting the time needed to implement the random values generated in the <cstdlib>. Also, I've also modify the int max_size of 100 and inside the main function I've modify the dataset into % 100 which divides it by 100 and find the remainder on it. Also, I've implemented the new function which displays the Unsorted Array.

Table 7-1. Array of Values for Sort Algorithm Testing

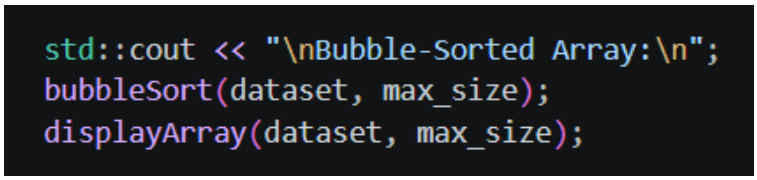
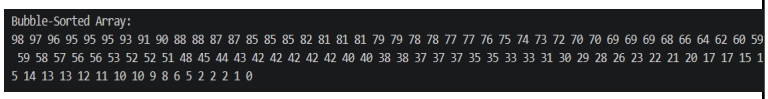
Code + Console Screenshot	  
Observations	In this code, I've implemented the algorithm for bubble sort which first loop is we find the current number arr[i] that we want to compare to the next number arr[j] in the array. Next loop, is we compare the arr[i] if it is greater than arr[j] then it will perform swapping; Otherwise, it will end the program. The time complexity is O(n^2) due to the nested loop when iterating the arr[i] and comparing it to the arr[j] to swap it or not.

Table 7-2. Bubble Sort Technique

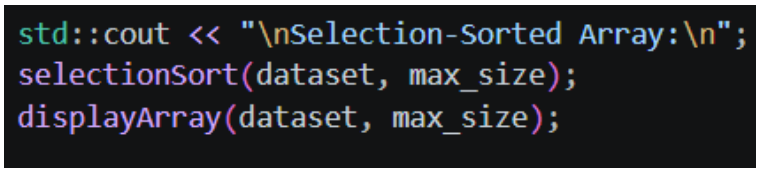
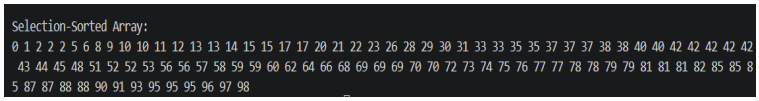
Code + Console Screenshot	  
Observations	In this code, I've implemented the Selection Sort Algorithm which it has a helper function which is the routine_smallest helper scans the array starting from index k through max_size - 1 to locate and return the index of the minimum element in that unsorted subarray. The selectionSort function loops from i = 0 to arrSize - 2, calling routine_smallest on each pass and swapping the minimum element found into its correct sorted position arr[i] using std::swap. The time complexity of this Algorithm is O(n^2) because it has a nested while for loops which is in routine_smallest function and for the selectionSort function.

Table 7-3. Selection Sort Algorithm

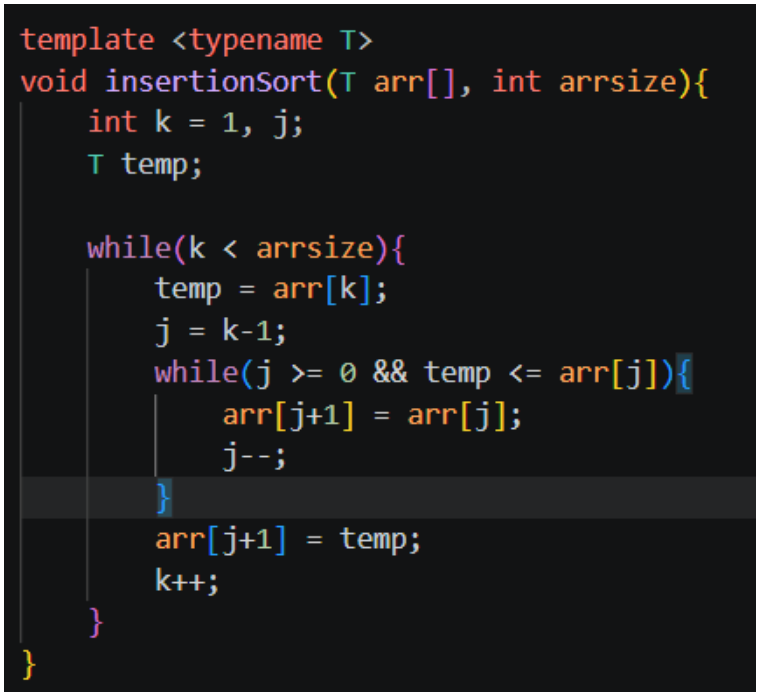
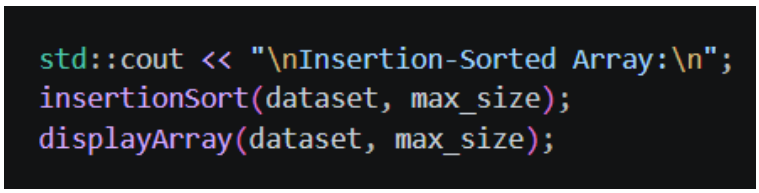
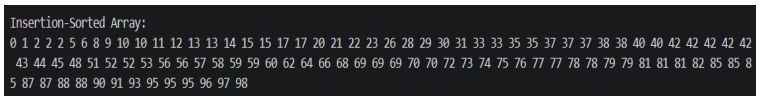
Code + Console Screenshot	  
Observations	In this code, I've implemented the algorithm in Insertion, which the outer while loop iterates through the array from index 1 to arrsize - 1, storing the key element in temp. The inner while loop shifts elements greater than or equal to temp one position to the right until the correct insertion index j + 1 is found which is similar in bubble sort but it is in descending order. The time complexity of this is O(n^2) because the nested while loops compare each element to every previous element in the sorted subarray to shift elements and place temp in its correct position.

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PSEUDOCODE OF THE ALGORITHM

SORTING ALGORITHM

```
ALGORITHM insertionSort(arr, arrsize)
   DECLARE k = 1
   DECLARE j AS INTEGER
   DECLARE temp AS TYPE OF arr

   WHILE k < arrsize DO
      SET temp = arr[k]
      SET j = k - 1

      WHILE j >= 0 AND temp <= arr[j] DO
         SET arr[j + 1] = arr[j]
         SET j = j - 1
      END WHILE

      SET arr[j + 1] = temp
      SET k = k + 1
   END WHILE
END FUNCTION
```

SEARCHING ALGORITHM

```
ALGORITHM llinearSearchArrVoteCount(dataset, arrSize, candidatesID, candidates)
   DECLARE voteCounter = 0

   FOR i FROM 0 TO arrSize - 1 DO
      IF dataset[i] EQUALS candidatesID THEN
         INCREMENT voteCounter BY 1
      END IF
   END FOR

   PRINT "Candidate ", candidatesID, " (", candidates[candidatesID - 1], ") received ", voteCounter, " votes."

   RETURN voteCounter
END FUNCTION
```

```cpp
#include <iostream>
#include "sortingAlgoSuppl.h"
#include <cstdlib>
#include <time.h>
#include <map>
#include <vector>
#include <string>

// max size = 100 values in the array will be from 1 to 5 only
const int max_size = 100;

int main(){
    int dataset[max_size];

    // List of candidates in array
    std::vector<std::string> candidates = {
        "Bo Dalton Capistrano",     // Candidate 1
        "Cornelius Raymon Agustín", // Candidate 2
        "Deja Jayla Bañaga",        // Candidate 3
        "Lalla Brielle Yabut",      // Candidate 4
        "Franklin Relano Castro"    // Candidate 5
    };

    // Seed the random number generator with the current time
    srand(time(0));

    // Fill array A[0...100] with values 1 to 5 only
    for (int i = 0; i < max_size; i++) {
        dataset[i] = (rand() % 5) + 1;
    }

    // Display the unsorted array of votes
    std::cout << "Unsorted Array of Votes:\n";
    displayArray(dataset, max_size);

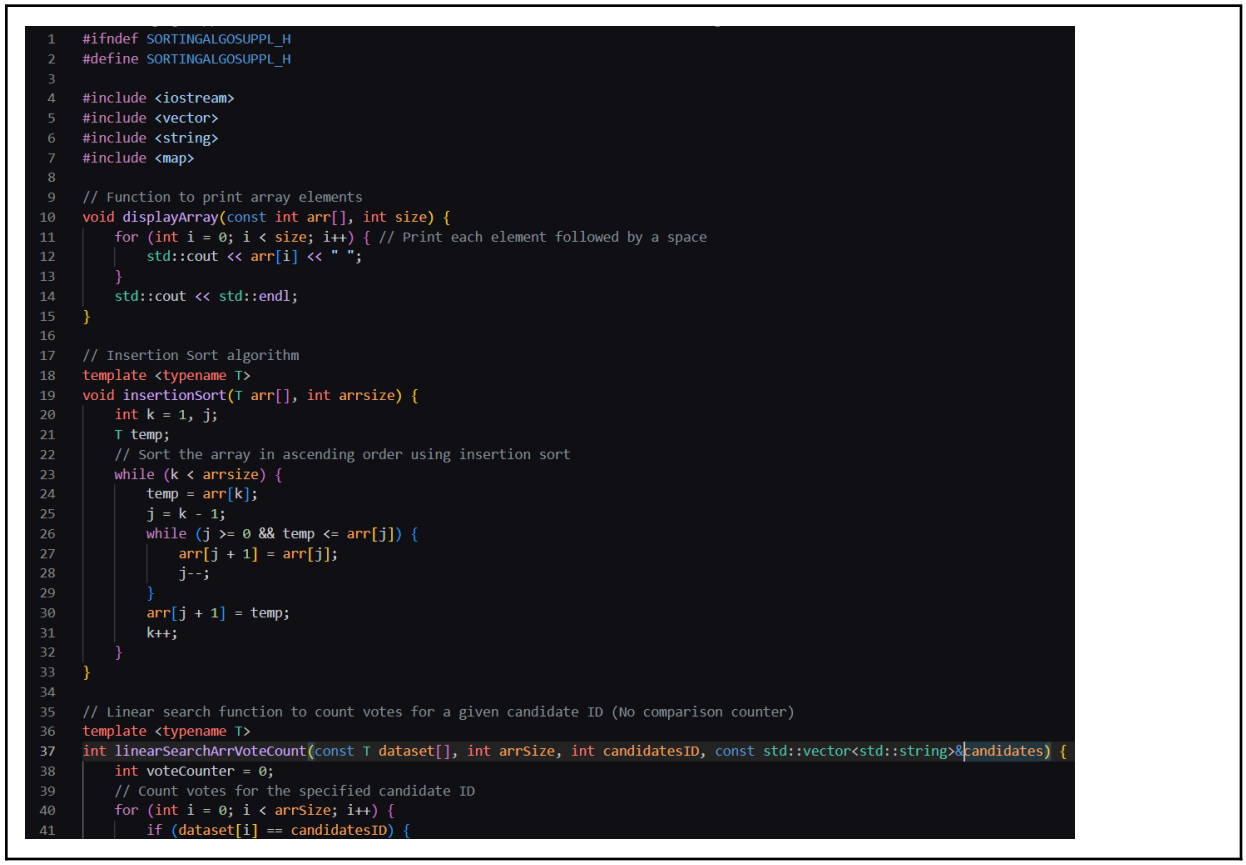
    // Sort the dataset using insertion sort
    std::cout << "\nInsertion-Sorted Array of Votes:\n";
    insertionSort(dataset, max_size);
    displayArray(dataset, max_size);

    // Count votes for each candidate using linear search
    std::map<int, int> voteCount;
    // Outputs individual candidate vote counts
    std::cout << "\n--- Individual Candidate Vote Counts (Linear Search) ---\n";
    for (int id = 1; id <= static_cast<int>(candidates.size()); id++) {
        voteCount[id] = linearSearchArrVoteCount(dataset, max_size, id, candidates);
    }

    // Display the winner or tied candidates
    displayVoteWinner(dataset, max_size, candidates);

    return 0;
}
```

mainSuppl.cpp code



```cpp
#ifndef SORTINGALGOSUPPL_H
#define SORTINGALGOSUPPL_H

#include <iostream>
#include <vector>
#include <string>
#include <map>

// Function to print array elements
void displayArray(const int arr[], int size) {
    for (int i = 0; i < size; i++) { // Print each element followed by a space
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}

// Insertion Sort algorithm
template <typename T>
void insertionSort(T arr[], int arrsize) {
    int k = 1, j;
    T temp;
    // Sort the array in ascending order using insertion sort
    while (k < arrsize) {
        temp = arr[k];
        j = k - 1;
        while (j >= 0 && temp <= arr[j]) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = temp;
        k++;
    }
}

// Linear search function to count votes for a given candidate ID (No comparison counter)
template <typename T>
int linearSearchArrVoteCount(const T dataset[], int arrSize, int candidatesID, const std::vector<std::string>&candidates) {
    int voteCounter = 0;
    // Count votes for the specified candidate ID
    for (int i = 0; i < arrSize; i++) {
        if (dataset[i] == candidatesID) {
```

```
            voteCounter++;
        }
    }
    // Output the vote count for the candidate
    std::cout << "Candidate " << candidatesID << " (" << candidates[candidatesID - 1]
              << ") received " << voteCounter << " votes.\n";
    // Return the total votes counted for the candidate
    return voteCounter;
}

// Function to find and display the winner or tied candidates
template <typename T>
void displayVoteWinner(const T dataset[], int arrSize, const std::vector<std::string>& candidates) {
    // Map to hold candidate IDs and their corresponding vote counts
    std::map<int, int> voteCount;

    // Tally votes from array
    for (int i = 0; i < arrSize; i++) {
        voteCount[dataset[i]]++;
    }

    int maxVotes = 0;

    // Find the maximum votes received by any candidate
    for (const auto& pair : voteCount) {
        if (pair.second > maxVotes) {
            maxVotes = pair.second;
        }
    }

    // Collect candidates that achieved maxVotes
    std::vector<int> winners;
    for (const auto& pair : voteCount) {
        if (pair.second == maxVotes && maxVotes > 0) {
            winners.push_back(pair.first);
        }
    }
```

```
    // Display the election results
    std::cout << "\n-------------ELECTION RESULTS-------------\n";
    if (winners.empty()) { // No votes were counted for any candidate
        std::cout << "No votes were counted for each candidate.\n";

    } else if (winners.size() == 1) { // Single winner
        int winnerID = winners[0];
        std::cout << "The winner of the election is Candidate " << winnerID
                  << " (" << candidates[winnerID - 1] << ") with " << maxVotes << " votes!!!\n";

    } else { // Tie situation
        std::cout << "It's a tie! No clear winner in the election.\n";
        std::cout << "The following candidates tied with " << maxVotes << " votes each:\n";
        for (int winnerID : winners) { // Display each tied candidate's ID and name
            std::cout << " - Candidate " << winnerID << " (" << candidates[winnerID - 1] << ")\n";
        }
    }
    std::cout << "\n-----------------------------------\n";
}

#endif
```

sortingAlgoSuppl.h code

```
Unsorted Array of Votes:
1 4 2 2 5 3 5 2 3 5 5 5 1 1 5 2 2 1 5 3 3 1 3 5 4 2 4 5 1 3 1 5 1 2 5 2 3 2 5 4 5 4 3 1 5 1 1 1 1 2 4 4 1 3 4 1 5 5 3 5 2 4 4 4 4 2 1 1 4 1 4 1 2 1 1 5 3 3 3 4
3 3 4 2 3 2 4 2 3 3 5 1 2 5 4 2 2 5 3 2

Insertion-Sorted Array of Votes:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5
5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5

--- Individual Candidate Vote Counts (Linear Search) ---
Candidate 1 (Bo Dalton Capistrano) received 22 votes.
Candidate 2 (Cornelius Raymon Agustín) received 20 votes.
Candidate 3 (Deja Jayla Bañaga) received 19 votes.
Candidate 4 (Lalla Brielle Yabut) received 18 votes.
Candidate 5 (Franklin Relano Castro) received 21 votes.

-------------ELECTION RESULTS-------------
The winner of the election is Candidate 1 (Bo Dalton Capistrano) with 22 votes!!!

-----------------------------------
```

Output Console

Output Console Showing Sorted Array

```
Unsorted Array of Votes:
4 5 4 4 2 2 1 3 3 1 1 4 2 1 1 3 4 2 2 3 2 5 4 1 1 3 2 5 2 5 3 2 1 2 1 5 4 3 3 4 1 1 4 2 5 4 3 5 5 2 4 1 3 3 3 2 2 1 4 3 4 2 1 4 1 4 4 2 1 5 2 1 4 2 1 1 5 1 2 5
2 5 4 3 2 5 5 4 5 2 3 1 4 3 2 4 3 5 3 1

Insertion-Sorted Array of Votes:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4
4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
```

```
Unsorted Array of Votes:
3 4 4 3 1 1 3 3 4 2 5 4 4 2 5 5 3 4 1 5 5 2 1 5 3 3 5 5 2 1 5 5 2 5 2 5 1 2 1 2 4 5 4 5 1 4 3 2 1 1 3 4 1 4 1 1 2 5 3 2 4 1 1 4 5 3 4 2 5 3 2 5 3 1 3 3 3 4 5 5
1 1 3 4 1 2 4 2 5 5 4 2 2 1 1 3 3 5 1 2

Insertion-Sorted Array of Votes:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5
5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
```

```
Unsorted Array of Votes:
5 3 5 4 5 3 4 5 4 3 4 1 3 5 2 4 5 5 2 2 2 3 1 1 3 1 4 1 1 1 2 2 3 3 3 1 2 1 3 1 4 1 4 5 1 4 3 3 3 3 5 4 3 2 1 4 2 5 3 1 3 5 2 4 1 3 2 1 3 3 5 4 3 2 3 4 5 1 5 5
5 4 4 5 4 4 3 5 2 3 4 3 5 5 2 1 3 2 3 3

Insertion-Sorted Array of Votes:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
```

Manual Count
--- Individual Candidate Vote Counts (Linear Search) --- Candidate 1 (Bo Dalton Capistrano) received 22 votes. Candidate 2 (Cornelius Raymon Agustín) received 23 votes. Candidate 3 (Deja Jayla Bañaga) received 18 votes. Candidate 4 (Lalla Brielle Yabut) received 21 votes. Candidate 5 (Franklin Relano Castro) received 16 votes.
--- Individual Candidate Vote Counts (Linear Search) --- Candidate 1 (Bo Dalton Capistrano) received 22 votes. Candidate 2 (Cornelius Raymon Agustín) received 18 votes. Candidate 3 (Deja Jayla Bañaga) received 19 votes. Candidate 4 (Lalla Brielle Yabut) received 18 votes. Candidate 5 (Franklin Relano Castro) received 23 votes.
--- Individual Candidate Vote Counts (Linear Search) --- Candidate 1 (Bo Dalton Capistrano) received 18 votes. Candidate 2 (Cornelius Raymon Agustín) received 15 votes. Candidate 3 (Deja Jayla Bañaga) received 28 votes. Candidate 4 (Lalla Brielle Yabut) received 19 votes. Candidate 5 (Franklin Relano Castro) received 20 votes.

Count Result of Algorithm
``` ------------ELECTION RESULTS------------ The winner of the election is Candidate 2 (Cornelius Raymon Agustín) with 23 votes!!! ---------------------------------- ```
``` ------------ELECTION RESULTS------------ The winner of the election is Candidate 5 (Franklin Relano Castro) with 23 votes!!! ---------------------------------- ```
``` ------------ELECTION RESULTS------------ The winner of the election is Candidate 3 (Deja Jayla Bañaga) with 28 votes!!! ---------------------------------- ```

Question: Was your developed vote counting algorithm effective? Why or why not?

Yes. Based from the Insertion Algorithm and the Linear Searching Algorithm, I have seen that the code balances well between the algorithmic structure and the accurate data processing to make sure the election results are reliable and accurate. The Insertion Sort nicely arranges the raw, unsorted data set into contiguous groups of identical votes, and the Linear Search systematically goes through the array for each candidate's exact vote count, avoiding skipping their records or index errors. Moreover, by combining these algorithms with a dynamic mapping and a vector-based formatting mechanism, the system is able to automatically detect single-winner victories and multi-candidate ties, thus providing an accurate, structured and error-tolerant solution to the small-scale vote counting problem.

WHY INSERTION SORT??

In the Insertion Sort Algorithm, I used it because it is very easy to sort the raw data into contiguous groups of identical votes as it is in ascending order and it is especially effective for small, nearly-sorted or bounded datasets like our 100 element vote array. Insertion Sort builds the final sorted array (or list) one item at a time by inserting each item into its correct position in the array (or list). It has a very low in-place memory overhead which is very efficient for small data set sizes (N=100). Also, grouping same candidate id's in a row makes the dataset structure visually clearer for linear search verification in making the searching algorithm precise and accurate.

CONCLUSION/LEARNINGS/ ANALYSIS	**In this Hands-On Activity, I managed to learn the fundamental sorting algorithms which can be used in distinct scenarios. Particularly, I observed how different sorting techniques can affect the organization of data and consequently the ease of its search. When reviewing the provided instructions in the procedures, I've learned that while Bubble sort utilizes elements' swapping and repetition to achieve the goal, Selection sort focuses on selecting the minimum value and placing it at the beginning. Meanwhile, I've learned that the use of the Insertion sort algorithm is the most effective for the given scenario in the supplementary activity due to its ability to operate with smaller sets of information. Through the additional voting counter example, I saw how the ability to sort votes in ascending order allows for a faster data searching in linear search algorithm, which outputs the accurate counting of votes and detection of winners and ties. Overall, I managed to create the required code snippets using templates, maps, and header files. However, I would improve my coding skills in terms of manipulating different kinds of libraries like (map and vector). Map library is used for efficient, dynamically key-indexed vote counting, while Vector library provides a flexible, dynamic storage of data for candidate names and potential tied winners. Also, I need to improve my code-organization skills to properly organize my arguments in the functions which improves my debugging skills in the C++ programming language.**

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **NONE**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **NONE**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **NONE**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.